Monorepo Setup with Turborepo

A Complete Guide to Building a Turborepo-based Monorepo with Next.js, React, and Bun

Author: Yaswanth Gudivada

Table of Contents

1 Introduction to Turborepo Monorepo ... 3
- 1.1 Running the Default Applications ... 3
- 1.2 How Turborepo Knows Which Projects to Run ... 3

2 Understanding the `turbo.json` Configuration ... 3
- 2.1 The `dependsOn` Field ... 4
- 2.2 The `outputs` Field and Caching ... 4
- 2.3 The `dev` Task Configuration ... 4

3 Adding a New React Application to the Monorepo ... 5
- 3.1 Can `react-app` Use `packages/ui` Without Declaring It as a Dependency? ... 5

4 Creating a Bun-based Express Backend ... 5
- 4.1 Initial Setup and the Parsing Error in Zed ... 5
- 4.2 Root Cause of the Error ... 6
- 4.3 How the Error Was Fixed ... 6
  - 4.3.1 Fix 1: Update `apps/backend/eslint.config.js` ... 6
  - 4.3.2 Fix 2: Update `apps/backend/tsconfig.json` ... 7

5 Centralizing TypeScript Configuration ... 7
- 5.1 Previous Local `tsconfig.json` (Before Centralization) ... 7
- 5.2 Refactored `tsconfig.json` (Extending from `@repo/typescript-config`) ... 8

6 Centralizing ESLint Configuration ... 8
- 6.1 TypeScript Declaration Error for `@repo/eslint-config/base` ... 10
  - 6.1.1 Why This Error Occurs ... 10
  - 6.1.2 How to Fix It ... 10

7 Cleaning Up `package.json` Dependencies ... 11
- 7.1 Backend `package.json` (Before Centralization) ... 11
- 7.2 Backend `package.json` (After Centralization) ... 11

8 Why the Backend Does Not Start with `bun run dev` ... 12

9 Creating a Shared `common` Package in `packages/` ... 13
- 9.1 Steps to Create `packages/common` ... 13
- 9.2 Example Content of `packages/common/src/index.ts` ... 13
- 9.3 Install Dependencies Globally ... 14

10 Using the `@repo/common` Package in the Backend ... 14

10.1 Warning: Always Declare the Dependency Explicitly 14
11 A Note on Bun vs. `tsc` Compiler 15
11.1 The Solution: Use a Dedicated Bundler 15
12 Summary 15

1 Introduction to Turborepo Monorepo

When we initialize a monorepo using the command `bunx create-turbo@latest`, Turborepo generates a predefined project structure for us. By default, this structure contains two primary folders at the root level:

- **`apps/`** — This folder holds runnable applications. When the project is first created, it contains two Next.js projects by default: one named `web` and the other named `docs`.
- **`packages/`** — This folder holds shared code that can be consumed by any of the applications inside the `apps/` folder. Examples of shared code include UI components, ESLint configurations, TypeScript configurations, and other reusable utilities.

The key idea behind this separation is that the code inside `packages/` is meant to be common across all applications. For example, a button component located at `packages/ui/src/button.tsx` can be imported and used by both the `docs` application and the `web` application inside the `apps/` folder. This promotes code reuse and prevents duplication across projects.

1.1 Running the Default Applications

Once the monorepo is set up, running the following command from the terminal will start all applications simultaneously:

```
bun run dev
```

By default, both projects start on different ports:

- `web` runs at `http://localhost:3000/`
- `docs` runs at `http://localhost:3001/`

1.2 How Turborepo Knows Which Projects to Run

In the root `package.json`, the `dev` script is defined as follows:

```
"dev": "turbo dev"
```

So, when we execute `bun run dev`, it internally runs `turbo dev`. A natural question arises: how does `turbo dev` know that it must run only the `web` and `docs` projects inside the `apps/` folder, and not any other folders?

The answer lies in the `turbo.json` configuration file present at the root of the monorepo. This file defines the tasks Turborepo is aware of and how they should be executed.

2 Understanding the `turbo.json` Configuration

The default `turbo.json` looks like this:

```
{
  "$schema": "https://turborepo.dev/schema.json",
```

```
  "ui": "tui",
  "tasks": {
    "build": {
      "dependsOn": ["^build"],
      "inputs": ["$TURBO_DEFAULT$", ".env*"],
      "outputs": [".next/**", "!.next/cache/**"]
    },
    "lint": {
      "dependsOn": ["^lint"]
    },
    "check-types": {
      "dependsOn": ["^check-types"]
    },
    "dev": {
      "cache": false,
      "persistent": true
    }
  }
}
```

2.1 The `dependsOn` Field

The expression `"dependsOn": ["^build"]` tells Turborepo that when the user runs `bun run build`, it should traverse into each workspace inside the `apps/` folder (such as `web` and `docs`) and execute their respective `build` scripts. The same logic applies to `lint` and `check-types`.

An important observation is that `web` and `docs` both have a `build` script defined in their own `package.json`, but `packages/ui` does not have one. This is intentional because shared UI packages typically do not need to be built separately — they are consumed directly as source.

2.2 The `outputs` Field and Caching

The line `"outputs": [".next/**", "!.next/cache/**"]` configures Turborepo's caching behavior. When `bun run build` is executed, Next.js generates a `.next/` folder containing the build output. Turborepo caches this `.next/` folder so that, on subsequent builds where no inputs have changed, it can restore the cached build output instead of recompiling everything from scratch.

However, the `.next/cache/` subdirectory is explicitly excluded (`!.next/cache/**`). This is because `.next/cache/` contains Next.js's own internal cache data, which is not necessary for the final build output and may lead to inconsistent caching behavior if included.

2.3 The `dev` Task Configuration

Inside the `dev` task, two important options are specified:

- **`"cache": false`** — This tells Turborepo not to cache the results of the `dev` script. The rationale is straightforward: the `dev` script runs a persistent development server that produces no durable outputs, so there is nothing meaningful to cache.
- **`"persistent": true`** — This informs Turborepo that `dev` is a long-running process (a development server that keeps running). Declaring this property ensures that Turborepo will not allow

other tasks to depend on the `dev` task, which would otherwise cause the dependency graph to hang indefinitely.

3 Adding a New React Application to the Monorepo

Inside the `apps/` folder, we can add additional applications. For example, creating a new React project named `react-app` and placing it inside `apps/` is straightforward. After adding this project, running `bun install` from the root ensures that the root `package.json` installs all necessary dependencies across the workspaces.

Once installed, executing `bun run dev` starts all three projects together: `web`, `docs`, and `react-app`.

3.1 Can `react-app` Use `packages/ui` Without Declaring It as a Dependency?

A common question arises here: if we try to import `packages/ui/button.tsx` from `react-app` **without** adding the following dependency declaration in `react-app`'s `package.json`:

```
"dependencies": {
  "@repo/ui": "*"
}
```

does it still work?

The answer is: **yes, it works — but only coincidentally**. The reason is that the root `node_modules/` folder contains a symlink at `node_modules/@repo/ui/` because the original starter projects (`web` and `docs`) declared `@repo/ui` as a dependency. As a result, `react-app` can also resolve `@repo/ui` through the hoisted root `node_modules/`.

However, this coincidental resolution is not reliable. If in the future we delete `web` and `docs` and create a fresh React project inside `apps/`, then running `bun install` will no longer cause `@repo/ui` to appear in the root `node_modules/` (because nothing explicitly depends on it). In that case, the import will fail unless we explicitly add `"@repo/ui": "*"` to the new project's `package.json`.

Best practice: Always declare your workspace dependencies explicitly in each application's `package.json`. Do not rely on implicit hoisted resolution.

4 Creating a Bun-based Express Backend

Next, let us create a backend application using Bun and Express, exposing a simple health-check endpoint.

4.1 Initial Setup and the Parsing Error in Zed

We initialize a new Bun project using:

```
bun init -y
```

This generates a default Bun project. We then create a `src/` folder and move `index.ts` inside it. We also add ESLint and Prettier configurations for this project specifically. After doing this, the following parsing error appears in the Zed editor:

```
Parsing error: ESLint was configured to run on
`/Users/yaswanthgudivada/Documents/Learnings/HarkiratSingh/Cohort-2/FSD/Programs/
week_16/week_16.2/monorepo-project-2/apps/backend/src/index.ts`
using `parserOptions.project`: <tsconfigRootDir>/tsconfig.json
However, that TSConfig does not include this file. Either:
- Change ESLint's list of included files to not include this file
- Change that TSConfig to include this file
- Create a new TSConfig that includes this file and include it in your
parserOptions.project
See the typescript-eslint docs for more info: https://tseslint.com/none-of-those-
tsconfigs-include-this-file
```

4.2 Root Cause of the Error

This is a very common issue in monorepo setups when using Zed (or VS Code) along with ESLint flat configurations and TypeScript.

The editor typically launches the ESLint language server from the **root** of the monorepo workspace — not from the specific application folder. When ESLint found the configuration file at `apps/backend/eslint.config.js`, it read this rule:

```
project: "./tsconfig.json"
```

Because ESLint was being executed from the root of the monorepo, it assumed `./tsconfig.json` was located at the **repository root** rather than inside the `apps/backend/` directory. When it then failed to match `src/index.ts` against this (incorrect) project path, it emitted the parsing error shown above.

Additionally, TypeScript can behave unreliably when locating project files across IDE boundaries if an explicit `"include"` array is missing from `tsconfig.json`.

4.3 How the Error Was Fixed

The fix required modifying two configuration files inside `apps/backend/`.

4.3.1 Fix 1: Update `apps/backend/eslint.config.js`

We added `tsconfigRootDir: import.meta.dirname` to the `typescript-eslint` `parserOptions`. This instructs ESLint to always resolve `./tsconfig.json` **relative to the directory where** `eslint.config.js` **lives**, regardless of where the editor started the language server from:

```
parserOptions: {
  ecmaVersion: "latest",
  sourceType: "module",
  project: "./tsconfig.json",
  tsconfigRootDir: import.meta.dirname, // <-- Added this
}
```

4.3.2 Fix 2: Update `apps/backend/tsconfig.json`

We explicitly added `"include": ["src/**/*"]` to the root of the configuration. This guarantees that the TypeScript language server correctly recognizes `src/index.ts` as part of the project being linted:

```
{
  "include": ["src/**/*"]
}
```

After making these changes, it may be necessary to restart the ESLint language server or reload the Zed window (for example, via the Command Palette, running `workspace: reload`, or simply closing and reopening the editor) for the changes to take effect. Once reloaded, the error should be completely gone.

5 Centralizing TypeScript Configuration

While the local fix described above works, it is not the ideal monorepo approach. In a proper Turborepo setup, TypeScript configuration should live in `packages/typescript-config/`, and each application should **extend** from that central configuration. Most of the `tsconfig.json` content is already provided there, so we should not duplicate it.

5.1 Previous Local `tsconfig.json` (Before Centralization)

Before refactoring, the backend's `tsconfig.json` contained the full configuration inline:

```
{
  "compilerOptions": {
    // environment setup & latest features
    "lib": ["esnext"],
    "target": "esnext",
    "module": "esnext",
    "moduleDetection": "force",
    "jsx": "react-jsx",
    "allowJs": true,

    // bundler mode
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "verbatimModuleSyntax": true,
    "noEmit": true,

    // best practices
    "strict": true,
    "skipLibCheck": true,
    "noFallthroughCasesInSwitch": true,
    "noUncheckedIndexedAccess": true,

    // some stricter flags (disabled by default)
    "noUnusedLocals": false,
    "noUnusedParameters": false,
```

```
    "noPropertyAccessFromIndexSignature": false
  },
  "include": ["src/**/*"]
}
```

5.2 Refactored `tsconfig.json` (Extending from `@repo/typescript-config`)

After refactoring to import from `packages/typescript-config/base.json`, the backend's `tsconfig.json` now looks like this:

```
{
  "extends": "@repo/typescript-config/base.json",
  "compilerOptions": {
    "lib": ["ESNext"],
    "target": "ESNext",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "moduleDetection": "force",
    "jsx": "react-jsx",
    "allowJs": true,
    "allowImportingTsExtensions": true,
    "verbatimModuleSyntax": true,
    "noEmit": true,
    "noFallthroughCasesInSwitch": true,
    "noUnusedLocals": false,
    "noUnusedParameters": false,
    "noPropertyAccessFromIndexSignature": false
  },
  "include": ["src/**/*"]
}
```

The `"extends"` key tells TypeScript to inherit all options from the shared base configuration, and any options specified locally will override or augment those inherited options.

6 Centralizing ESLint Configuration

The same pattern of centralization applies to ESLint. By default, the backend's `eslint.config.js` looked like this (a fully self-contained configuration):

```
// Universal ESLint + Prettier config for Bun projects
import js from "@eslint/js";
import globals from "globals";
import prettier from "eslint-config-prettier";

let tsPlugin;
try {
  tsPlugin = await import("@typescript-eslint/eslint-plugin");
```

```
} catch {}

const baseConfig = [
  js.configs.recommended,
  {
    languageOptions: {
      ecmaVersion: "latest",
      sourceType: "module",
      globals: {
        ...globals.node,
        Bun: "readonly",
      },
    },
    rules: {
      eqeqeq: ["error", "always"],
      "no-var": "error",
      "prefer-const": "warn",
      "no-mixed-spaces-and-tabs": "error",
      "no-trailing-spaces": "warn",
      "object-curly-spacing": ["error", "always"],
      "no-unused-vars": "warn",
      "no-console": "off",
    },
  },
];

if (tsPlugin) {
  baseConfig.push({
    files: ["**/*.ts", "**/*.tsx"],
    languageOptions: {
      parser: (await import("@typescript-eslint/parser")).default,
      parserOptions: {
        ecmaVersion: "latest",
        sourceType: "module",
        project: "./tsconfig.json",
      },
    },
    plugins: {
      "@typescript-eslint": tsPlugin.default,
    },
    rules: {
      ...tsPlugin.default.configs.recommended.rules,
      "@typescript-eslint/no-unused-vars": "warn",
    },
  });
}

baseConfig.push(
  {
    ignores: ["node_modules", "dist", "build", ".next", ".turbo"],
  },
```

```
  prettier
);

export default baseConfig;
```

After importing from `packages/eslint-config/base`, the file becomes much shorter and easier to maintain:

```
import { config } from "@repo/eslint-config/base";

/** @type {import("eslint").Linter.Config[]} */
export default [
  ...config,
  {
    languageOptions: {
      globals: {
        Bun: "readonly",
      },
    },
  },
];
```

6.1 TypeScript Declaration Error for `@repo/eslint-config/base`

After performing the ESLint import refactor, the following TypeScript error appeared:

```
Could not find a declaration file for module '@repo/eslint-config/base'.
'/Users/yaswanthgudivada/.../packages/eslint-config/base.js' implicitly has an 'any'
type.
  Try `npm i --save-dev @types/repo__eslint-config` if it exists or add a new
  declaration (.d.ts) file containing `declare module '@repo/eslint-config/base';`
```

6.1.1 Why This Error Occurs

The reason for this error is that the `@repo/eslint-config` package exports its configurations as **pure JavaScript files** (`base.js`, `next.js`, and so on), without any corresponding TypeScript declaration (`.d.ts`) files. Since the consuming projects check types (or run TypeScript in strict mode), TypeScript complains about the missing type declarations.

6.1.2 How to Fix It

The proper fix is to add the missing declaration files directly to the `packages/eslint-config/` package. These declaration files declare the exported configuration variables as arrays of `Linter.Config` objects:

`packages/eslint-config/base.d.ts`

```
import type { Linter } from "eslint";
export const config: Linter.Config[];
```

`packages/eslint-config/next.d.ts`

```
import type { Linter } from "eslint";
export const nextJsConfig: Linter.Config[];
```

`packages/eslint-config/react-internal.d.ts`

```
import type { Linter } from "eslint";
export const config: Linter.Config[];
```

With these `.d.ts` files in place, TypeScript now has proper type information for the exports from `@repo/eslint-config`, and the error is completely suppressed in the editor.

7 Cleaning Up `package.json` Dependencies

Since ESLint and TypeScript configurations are now provided through shared workspace packages, we can remove the individual dependencies that were previously installed locally in the backend.

7.1 Backend `package.json` (Before Centralization)

```
{
  "name": "backend",
  "module": "src/index.ts",
  "type": "module",
  "private": true,
  "devDependencies": {
    "@types/bun": "latest",
    "eslint": "^9.33.0",
    "@eslint/js": "^9.33.0",
    "eslint-config-prettier": "^9.1.0",
    "globals": "^15.9.0",
    "prettier": "^3.3.3",
    "@typescript-eslint/parser": "^8.11.0",
    "@typescript-eslint/eslint-plugin": "^8.11.0"
  },
  "peerDependencies": {
    "typescript": "^5"
  },
  "scripts": {
    "lint": "eslint .",
    "lint:fix": "eslint . --fix",
    "lint:json": "eslint . --format json",
    "format": "prettier --write ."
  }
}
```

7.2 Backend `package.json` (After Centralization)

After importing shared configurations from `packages/`, many dependencies are no longer needed locally — they are provided transitively through the shared packages. The simplified `package.json` now looks like this:

```
{
  "name": "backend",
  "module": "src/index.ts",
  "type": "module",
  "private": true,
  "devDependencies": {
    "@types/bun": "latest",
    "eslint": "^9.33.0",
    "prettier": "^3.3.3",
    "@repo/eslint-config": "workspace:*",
    "@repo/typescript-config": "workspace:*"
  },
  "peerDependencies": {
    "typescript": "^5"
  },
  "scripts": {
    "lint": "eslint .",
    "lint:fix": "eslint . --fix",
    "lint:json": "eslint . --format json",
    "format": "prettier --write ."
  },
  "dependencies": {
    "@types/express": "^5.0.6",
    "express": "^5.2.1"
  }
}
```

Notice the use of `workspace:*` protocol for `@repo/eslint-config` and `@repo/typescript-config`. This tells Bun (or npm/yarn/pnpm) to resolve these packages from within the monorepo workspace, not from a remote registry.

8 Why the Backend Does Not Start with `bun run dev`

After setting up the backend, running `bun run dev` from the root starts the `web`, `docs`, and `react-app` applications — but **not** the `backend`. Why?

The reason is that Turborepo looks for a `dev` script in each workspace's `package.json`. If a workspace does not define a `dev` script, Turborepo has nothing to run for it.

At this point, the backend's `scripts` section looked like this:

```
"scripts": {
  "lint": "eslint .",
  "lint:fix": "eslint . --fix",
  "lint:json": "eslint . --format json",
  "format": "prettier --write ."
}
```

There is no `dev` entry, so Turborepo silently skips the backend. The fix is to add a `dev` script:

```
"scripts": {
  "dev": "bun --watch src/index.ts",
  "lint": "eslint .",
  "lint:fix": "eslint . --fix",
  "lint:json": "eslint . --format json",
  "format": "prettier --write ."
}
```

Here, `bun --watch src/index.ts` starts the backend using Bun's built-in file watcher, which automatically restarts the server on code changes. After this change, `bun run dev` from the root correctly starts all four projects: `web`, `docs`, `react-app`, and `backend`.

9 Creating a Shared `common` Package in `packages/`

The backend is a Bun-based project and does not need UI components from `packages/ui/`. However, there may be common utilities, constants, or URLs that are shared between the backend and frontend applications. To support this, we create a new shared package inside `packages/` called `common`.

9.1 Steps to Create `packages/common`

1. Create a new folder at `packages/common/`.
2. Initialize a Bun project inside it and set up Express if needed.
3. Configure `eslint.config.js` and `tsconfig.json` to extend from `packages/typescript-config` and `packages/eslint-config`, following the same centralization pattern described earlier.
4. In `packages/common/package.json`, rename the package from `"name": "common"` to `"name": "@repo/common"`. This is essential because only namespaced package names (prefixed with `@repo/`) are correctly resolved by workspaces.
5. Add an `exports` field to `package.json` so that applications in the `apps/` folder can see and import the package's files:

```
"exports": {
  "./*": "./src/*.ts"
}
```

The `./*` mapping means any subpath import (e.g., `@repo/common/index`) will be resolved to `./src/index.tsx` (or whichever file matches the `*` glob).

9.2 Example Content of `packages/common/src/index.ts`

```
export const VALUE = 'Yaswanth Gudivada';

// This URL may be needed by both backend and frontend,
// so it is kept in the shared common package.
// Path: packages/common
export const BACKEND_URL = 'https://api.google.com';
```

This `BACKEND_URL` constant, for example, would typically be needed by both backend and frontend code, which is why it makes sense to centralize it here in `packages/common`.

9.3 Install Dependencies Globally

After creating the new package, run the following command from the root of the monorepo to install all dependencies and link the new workspace:

```
bun install
```

10 Using the `@repo/common` Package in the Backend

Now let us try to import from `@repo/common` inside the backend. Edit `apps/backend/src/index.ts` as follows:

```
import express from 'express';
import { VALUE } from '@repo/common/index';

const app = express();
const PORT = 5050;

app.use(express.json());
console.log(`Name of the VALUE is: ${VALUE}`);

app.get('/healthy', (_req, res) => {
  res.status(200).json({
    message: 'Server is healthy',
  });
});

app.listen(PORT, () => {
  console.log(`Server is listening on http://localhost:${PORT}`);
});
```

When this backend is run, the console prints:

```
Name of the VALUE is: Yaswanth Gudivada
```

This confirms that the import from `@repo/common` is resolving correctly and working as expected.

10.1 Warning: Always Declare the Dependency Explicitly

Once again, it is important to stress an observation that was made earlier. At this point in the setup, `backend/package.json` does **not** yet include `"@repo/common": "*"` in its `dependencies`. Despite that, the import still works — because `@repo/common` happens to be present in the root `node_modules/@repo/` directory (because other workspaces depend on it, or because Bun's workspace resolution picked it up).

This is the **same coincidental resolution pattern** that was discussed earlier regarding `@repo/ui`. While it works, it is not reliable. If the packages that happen to hoist `@repo/common` are removed in the future, the backend's import will suddenly break.

Always explicitly declare workspace dependencies in each consuming project's `package.json`, like this:

```
"dependencies": {
  "@repo/common": "*",
  "@types/express": "^5.0.6",
  "express": "^5.2.1"
}
```

This guarantees reliable, deterministic resolution regardless of what other workspaces contain.

11 A Note on Bun vs. `tsc` Compiler

In the setup described above, everything works smoothly because we are using Bun as the runtime. Bun has first-class TypeScript support built in — it handles the TypeScript-to-JavaScript conversion transparently at runtime, which means constructs like:

```
export const VALUE = 'Yaswanth Gudivada';
```

work without any additional build step.

However, if we were to use the standard TypeScript compiler (`tsc`) instead of Bun, we would encounter **export resolution issues**. The compiled output would not correctly resolve such exports across workspace boundaries without additional build configuration.

11.1 The Solution: Use a Dedicated Bundler

To solve these issues when working outside of Bun's runtime, we should use a dedicated bundler such as:

- **`tsup`** — A zero-config TypeScript bundler that produces correctly exported JavaScript (and declaration files).
- **`esbuild`** — An extremely fast JavaScript bundler that can also handle TypeScript and produce correctly structured output.

These tools take care of bundling the package's TypeScript source into a form that can be consumed reliably by any Node.js (or other) runtime, resolving the export issues that arise when using `tsc` alone.

12 Summary

This guide has walked through the complete lifecycle of setting up a Turborepo-based monorepo, covering:

- Initializing a monorepo with `bunx create-turbo@latest` and understanding the default folder structure (`apps/` and `packages/`).
- Understanding how Turborepo orchestrates tasks via `turbo.json`, including the meaning of `dependsOn`, `outputs`, `cache`, and `persistent`.
- Adding new applications (React, Bun-based Express backend) to the monorepo.
- Diagnosing and fixing a common ESLint + TypeScript parsing error that occurs in editors like Zed and VS Code.
- Centralizing TypeScript and ESLint configuration into shared packages (`@repo/typescript-config`, `@repo/eslint-config`) and extending them from individual applications.
- Creating TypeScript declaration files (`.d.ts`) to resolve implicit `any` errors for shared JavaScript-only configuration packages.
- Ensuring that every application defines a `dev` script so Turborepo can run it.
- Creating a shared `@repo/common` package for code used by both backend and frontend applications, and configuring its `exports` field correctly.
- Understanding the importance of **explicitly declaring workspace dependencies** in each project's `package.json`, rather than relying on coincidental hoisting.
- Understanding the runtime differences between Bun and `tsc`, and knowing when to reach for a dedicated bundler like `tsup` or `esbuild`.

Following this layered, explicit, and centralized approach keeps a monorepo maintainable, reliable, and easy to reason about as it grows.